

Retrieval-Augmented Generation of Ontologies from Relational Databases

Mojtaba Nayyer¹, Athish A Yog¹, Nadeen Fathallah¹, Ratan Bahadur Thapa¹, Hans-Michael Tautenhahn³, Anton Schnurpe³, and Steffen Staab^{1,2}

¹ Institute for Artificial Intelligence, University of Stuttgart, Stuttgart, Germany

² University of Southampton, Southampton, UK

³ Universitätsklinikum Leipzig, Leipzig, Germany
mojtaba.nayyeri@ki.uni-stuttgart.de

Abstract. Transforming relational databases into knowledge graphs with enriched ontologies enhances semantic interoperability and unlocks advanced graph-based learning and reasoning over data. However, previous approaches either demand significant manual effort to derive an ontology from a database schema or produce only a basic ontology. We present RIGOR—Retrieval-augmented Iterative Generation of RDB Ontologies—an LLM-driven approach that turns relational schemas into rich OWL ontologies with minimal human effort. RIGOR combines three sources via RAG—the database schema and its documentation, a repository of domain ontologies, and a growing core ontology—to prompt a generative LLM for producing successive, provenance-tagged “delta ontology” fragments. Each fragment is refined by a *judge-LLM* before being merged into the core ontology, and the process iterates table-by-table following foreign key constraints until coverage is complete. Applied to real-world databases, our approach outputs ontologies that score highly on standard quality dimensions such as accuracy, completeness, conciseness, adaptability, clarity, and consistency, while substantially reducing manual effort.

Keywords: Relational database · Ontology · Large language models.

1 Introduction

Relational databases [49] are crucial in data management due to their structured framework, offering a robust, flexible, and user-friendly method for querying and manipulating data. They are particularly adept at handling complex queries efficiently and remain the backbone of enterprise applications in healthcare, finance, e-commerce, government, and other sectors.

However, it is difficult to pose semantic queries [49] to access database content and integrate relational databases. Transforming a relational database into a knowledge graph (KG) [39] with a well-defined and enriched ontology can promote data *sharing* and *integration* [67] and enhance *learning* [56] and *reasoning* capabilities [17,69].

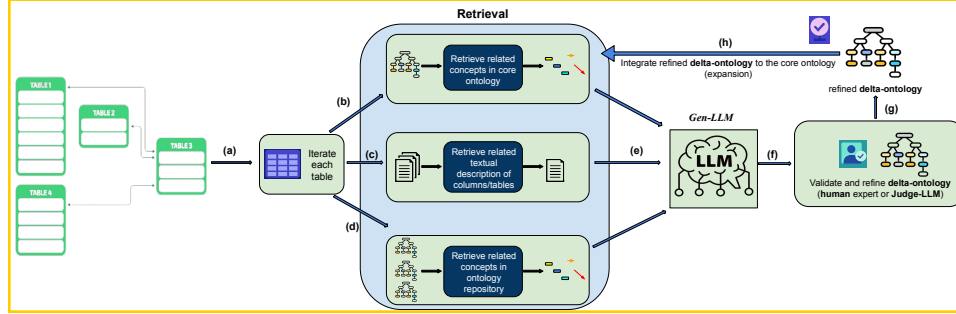


Fig. 1: Overview of our proposed iterative pipeline: (a) traverse tables following foreign key constraints; (b) retrieve relevant concepts from the core ontology using RAG; (c) retrieve related textual descriptions using RAG; (d) retrieve related concepts and properties from an ontology repository using RAG; (e) construct a prompt; (f) generate a delta-ontology using *Gen-LLM*; (g) refine the delta-ontology with either a human expert or a *Judge-LLM*; (h) integrate the refined delta-ontology into the core ontology to expand its coverage.

Creating ontologies from databases has required significant manual effort by domain experts or the use of semi-automated tools. These approaches rely on database schema (e.g., table and column names) only and lack access to external knowledge. As a result, they produce basic ontologies that fail to align with existing vocabularies or human-defined semantics [9]. Consequently, the resulting knowledge graphs may syntactically represent the data but lack the representative vocabulary and precise axiomatic definitions required for effective use by humans and machine reasoners.

Large language models (LLMs) [71] have recently shown a remarkable ability to comprehend text, capture and encode conceptual knowledge [70], perform algorithmic reasoning [72], and generate structured outputs by following instructions [36,35]. Models such as OpenAI’s ChatGPT (GPT-4) [57] and open-source models like Meta AI’s LLaMA 3 [20] and DeepSeek-V3 [24] have shown near-human performance on many knowledge-intensive tasks. LLMs have recently been used for ontology generation from text [6,54,44,47,33], but to the best of our knowledge, no work has yet explored the use of LLMs for ontology generation from relational databases.

We introduce RIGOR, an iterative Retrieval-Augmented Generation (RAG) pipeline that utilizes LLMs to systematically convert relational databases into OWL ontologies (see Figure 1). RIGOR incrementally processes the database schema table-by-table, guided by foreign-key relationships. Iteratively, it retrieves a table schema and relevant textual documentation—natural language descriptions of that table’s column names. It also retrieves associated concepts, object properties, and data properties from both an expanding core ontology

and external domain ontologies (e.g., BioPortal⁴ containing over 1000 ontologies and 15 million medical-domain classes). Using the retrieved context, a generative LLM (*Gen-LLM*) produces small ontology fragments (delta ontologies) for the given table schema, consisting of classes, objects, data properties, and attributes that extend or align with the core ontology. Each fragment is validated and refined through feedback from either human reviewers or an auxiliary *Judge-LLM* [23] before it is integrated into the core ontology. By selectively retrieving only the most relevant schema and ontology context at each step, RIGOR limits unnecessary information, reducing hallucinations and ensuring the resulting ontology remains accurate and semantically aligned with the original database.

We evaluate our approach through a comprehensive set of experiments on real-world medical relational databases, comparing our RIGOR pipeline against baseline and non-iterative ontology generation methods. We evaluate ontology quality using various methods, such as logical consistency checks semantic similarity analysis, and expert judgment, either human or via an independent *judge-LLM*. Our results show that RIGOR consistently outperforms competing methods, achieving significantly higher scores in accuracy, completeness, conciseness, adaptability, clarity, consistency, as well as semantic alignment with the underlying database schemas.

2 Related Work

2.1 Ontology Extraction from Relational Databases.

The semantic web and data integration communities have a long history of converting relational data to semantic representations [65,29,42,5]. The *relational-to-ontology mapping* problem was formalized in early work with the goal of developing an ontology that faithfully reflects the database content [5].

The W3C Direct Mapping and R2RML standards treat each table row as an RDF resource, providing a baseline knowledge graph that mirrors the physical schema and lacks higher-level abstraction or domain alignment [60,4,13]. BootOX automates relational-to-OWL mappings (table $\rightarrow$ class, foreign key $\rightarrow$ object property, etc.) and can invoke LogMap to align the resulting ontology with a chosen domain vocabulary, thereby accelerating ontology-based data-access solutions [26].

Other tools and approaches (e.g., Karma [28], IncMap [51], MIRROR [41], D2RQ [64]) have been evaluated in a benchmark for automatic relational-to-ontology mapping generation called RODI [50]. The RODI benchmark [50] evaluates how well systems like BootOX generate mappings that support correct query answering across domains (e.g., conference, geography, oil and gas). While effective for simple cases, results showed that complex scenarios with significant schema-ontology mismatches remain challenging.

Unlike prior works based on heuristics or string matching, our approach uses RAG-LLMs for semantic understanding and reasoning [70,72] to bridge the gap between database schemas and ontologies via contextualized semantic mapping.

⁴ <https://bioportal.bioontology.org/>

Ontology learning from databases using machine learning or additional metadata has been the subject of more recent studies [39]. For example, [9] suggests a technique for learning ontologies from RDBs that uses metrics for factual (ABox) and conceptual (TBox) quality to assess the approach. It introduces new heuristics to extract taxonomy and part-whole relations from the analysis of foreign keys and data patterns. They and others emphasize how crucial it is to assess the generated ontologies’ instance correctness and schema coverage. These efforts have a number of shortcomings, including a) relying on a static predefined mapping rule, which restricts generalization; b) assuming pre-cleared and normalized data while disregarding unstructured data; and c) lacking alignment with external ontologies or reuse. We refer to these surveys [49,46,49,39] for a comprehensive overview of other related works.

2.2 LLMs for Ontology and KG Generation.

Recent studies show that LLMs can perform automated knowledge engineering, including extracting structured knowledge and assisting in ontology generation from text [38,1,37,63,7,45,62,48,34,58,8,11,30,61,59,14,16]. [7] explored zero-shot LLMs for ontology learning tasks—term typing, taxonomy discovery, and relation extraction—finding limited success without fine-tuning. However, their approach relied on basic prompting, without advanced techniques like chain-of-thought or RAG. OntoKGen [1] addressed this by using GPT-4 with iterative, user-in-the-loop prompting to assist experts in building ontologies from technical documents, demonstrating LLMs’ potential in suggesting ontology elements while preserving human oversight.

Parallel to these, benchmarks like Text2KGBench [44] and OntoLLMBench [43] have been introduced to systematically evaluate LLMs in knowledge graph creation, fact extraction, and ontology population from text. While our work aligns with these efforts, it differs by focusing on structured inputs (database schemas) rather than unstructured text. Recent work [10] showed GPT-4 can map database schemas to semantics. We extend this with an RAG-based pipeline for formal OWL ontology generation.

2.3 Retrieval-Augmented Generation (RAG).

The idea of enhancing generative models with retrieval gained attention in open-domain QA, where RAG [32] improved factual accuracy by retrieving relevant Wikipedia passages. Later works [19] showed that providing relevant context via RAG reduces hallucinations in tasks like long-form QA, conversation, and code generation. RAG approaches can be classified into several types [19]: *naive RAG* (simple similarity-based retrieval added to prompts), *advanced RAG* (with query rewriting and reranking), *iterative RAG* (multi-step retrieval and selection), *recursive RAG* (breaking queries into sub-problems for iterative retrieval and generation), and *adaptive RAG* (dynamically switching between iterative and recursive strategies). In addition, RAG approaches can also be categorized based on the data type, such as text-based RAG, GraphRAG, multimodal RAG,

hybrid RAG (combining different types of input data), and structured data RAG (retrieving and generating based on structured data sources, such as databases or tables). In our work, we choose an **advanced hybrid recursive RAG**.

2.4 Competency Question Generation

Competency Questions (CQs) are natural language queries used to verify if an ontology captures the required domain knowledge [21]. Traditionally, formulating CQs has been manual and labor-intensive, supported by templates and controlled language patterns that still require significant expert effort [27]. Recently, LLMs have been used to automate CQ generation from ontology triples [2,55], domain texts [3], and knowledge graphs [12], showing promising results in producing contextually relevant CQs. Building on recent LLM-as-judge approaches [22,23], we use LLMs both to generate CQs and to evaluate whether the generated ontology meets these requirements. This automated evaluation helps verify consistency, coverage, and completeness, serving as a practical alternative to exhaustive human review.

3 Methodology

This section presents our novel approach called RIGOR (Retrieval Iterative Augmented Generation of RDB-Ontologies), which operates on the relational-database schema (RDB-schema). RIGOR’s complete workflow is illustrated in Figure 1: RIGOR builds the ontology by iteratively traversing the tables of the source RDB-Schema one by one, following foreign keys (Figure 1-(a)) and using a generative LLM (Gen-LLM). RIGOR extracts relevant context for every table (cf. Section 3.2) from (i) the expanding core ontology (Figure 1-(b)), (ii) formal schema definition as well as natural language descriptions of columns and tables (Figure 1-(c)), and (iii) external ontology sources (Figure 1-(d)).

With the retrieved information, the system constructs a prompt that includes the schema and contextual details of the table (cf. Figure 1-(e) and Section 3.3). This prompt is then passed to a Gen-LLM to generate an ontology fragment specific to the current table, referred to as the *delta-ontology* (Figure 1-(f)).

Before integrating this delta-ontology into the continuously expanding core ontology (Figure 1-(h) and cf. Section 3.5), it undergoes verification and refinement, either automatically (via a *judge-LLM*) or manually by a domain expert (cf. Section 3.4, Figure 1-(f)). This ensures semantic consistency and correctness of the generated ontology fragments.

The outcome of this iterative method is a comprehensive OWL 2 DL ontology that reflects the relational database structure, enriched by domain knowledge and aligned with ontological best practices.

3.1 Formalization of Data Structures

We formalize the RIGOR pipeline’s data structures—relational databases, text descriptions, ontologies, and intermediate artifacts—summarized in Table 5.

Relational Database Schema We denote the relational database schema by $\mathcal{S} = (\mathcal{R}, \mathcal{K}, \mathcal{F})$, where:

- $\mathcal{R}$ is the set of relations (i.e., tables) in the database.
- $\mathcal{K}$ is a function that maps each relation $r \in \mathcal{R}$ to its set of primary key attributes: $\mathcal{K}(r)$.
- $\mathcal{F}$ is a function that maps each relation $r \in \mathcal{R}$ to its set of attributes: $\mathcal{F}(r)$, where each attribute is given by its column *name* together with its *datatype*.

Each relation $r \in \mathcal{R}$ is characterized by its name, its attribute set $\mathcal{F}(r)$, its primary key $\mathcal{K}(r)$, and the set of attributes that participate in foreign-key constraints $\mathcal{F}_k(r) \subseteq \mathcal{F}(r)$.

Textual Descriptions of Tables and Columns Many database schemata come with natural language documentation. We formalize these as follows.

Let $\mathcal{D} \subset \mathcal{T}_{\text{Doc}}$ be the corpus of natural-language descriptions available for the database schema (e.g. comments in the DDL, a data dictionary, or README files). We define two *retrieval functions* that access this corpus:

- $\text{docTable} : \mathcal{R} \rightarrow \mathcal{D}$ returns the textual description of a relation $r \in \mathcal{R}$: $\text{docTable}(r) \in \mathcal{D}$.
- $\text{docAttr} : \{(r, a) \mid r \in \mathcal{R}, a \in \mathcal{F}(r)\} \rightarrow \mathcal{D}$ returns the textual description of an attribute a in relation r : $\text{docAttr}(r, a) \in \mathcal{D}$.

Ontology Representation We adopt OWL 2 DL as our target representation language [66]. At iteration i , the expanding core ontology (shown in the top part of the retrieval block in Figure 1) is denoted by $\mathcal{O}_i = (\mathcal{C}_i, \mathcal{P}_i, \mathcal{A}_i, \mathcal{M}_i)$, where

- $\mathcal{C}_i$ is the set of class identifiers (concept names).
- $\mathcal{P}_i$ is the set of properties, subdivided into object properties $\mathcal{P}_i^{\text{obj}}$ and data properties $\mathcal{P}_i^{\text{data}}$.
- $\mathcal{A}_i$ is the set of OWL 2 DL axioms, including class hierarchies, property domain and range assertions, and property characteristics.
- $\mathcal{M}_i$ is the set of annotation assertions, used for non-logical metadata such as labels, comments, and provenance information.

Ontology fragments produced by the *Gen-LLM* are expressed in OWL 2 DL using named classes, subclass axioms, existential restrictions, and property assertions. We serialize them in OWL 2 Manchester Syntax[25] because (i) its keyword-based, sentence-like format is far more readable for humans than Turtle or RDF/XML, and (ii) that same simple natural-language style makes it easier for large language models to generate and parse during prompt construction.

External Ontology Repository We leverage a repository of external ontologies to enrich the contextual information used during ontology generation. We formalize the external ontology repository as a set of n ontologies $\mathcal{E} = \{\mathcal{O}^{(1)}, \mathcal{O}^{(2)}, \dots, \mathcal{O}^{(n)}\}$ where each $\mathcal{O}^{(i)} = (\mathcal{C}^{(i)}, \mathcal{P}^{(i)}, \mathcal{A}^{(i)}, \mathcal{M}^{(i)})$ represents an individual ontology.

Delta-Ontology Fragment For each relation $r \in \mathcal{R}$, the Gen-LLM generates a delta-ontology fragment, denoted as $\Delta\mathcal{O}_r = (\mathcal{C}_r, \mathcal{P}_r, \mathcal{A}_r, \mathcal{M}_r)$, which captures the ontology elements derived from r and its contextual information (schema and textual descriptions). These fragments are iteratively integrated into $\mathcal{O}_i$.

Target Output: OWL Ontology The output of the RIGOR pipeline is a consolidated ontology $\mathcal{O}_{\text{final}} = \mathcal{O}_T$ after processing all relations in $\mathcal{R}$. $\mathcal{O}_{\text{final}}$ must satisfy:

- **Schema Coverage:** Every table $r \in \mathcal{R}$ is represented by one or more OWL classes.
- **Semantic Alignment:** Attributes and foreign keys in $\mathcal{F}(r), \mathcal{F}_k(r)$ are modeled as data properties/classes or object properties, respectively.
- **Provenance Annotations:** Every ontology element includes metadata referencing its source table or documentation.
- **External Alignment:** Wherever applicable, ontology elements are linked to external domain ontologies via `owl:sameAs` or `rdfs:subClassOf`.

Lexical views (1) **Single ontology.** For any ontology $\mathcal{O} = (\mathcal{C}, \mathcal{P}, \mathcal{A}, \mathcal{M})$, $\mathcal{P} = \mathcal{P}^{\text{obj}} \cup \mathcal{P}^{\text{data}}$, define its *lexical view*

$$\begin{aligned} \text{Lex}(\mathcal{O}) = & \underbrace{\left\{ \lambda \mid \exists e \in \mathcal{C} \cup \mathcal{P}^{\text{obj}} \cup \mathcal{P}^{\text{data}} : \langle e, \text{rdfs:label}, \lambda \rangle \in \mathcal{M} \right\}}_{\text{labels of classes \& properties}} \\ & \cup \underbrace{\left\{ \lambda \mid \lambda \in \mathcal{M} \wedge \lambda \text{ is a string literal} \right\}}_{\text{other textual annotations}}. \end{aligned}$$

(2) **Relational schema.**

$$\text{Lex}(\mathcal{S}) := \{ \text{name}(r) \mid r \in \mathcal{R} \} \cup \{ \text{name}(a) \mid r \in \mathcal{R}, a \in \mathcal{F}(r) \}.$$

Universe of texts. Let $\mathcal{E} = \{\mathcal{O}^{(1)}, \dots, \mathcal{O}^{(n)}\}$ and let $\text{docTable}, \text{docAttr}$ be the documentation functions. Then

$$\mathcal{X} := \text{Lex}(\mathcal{S}) \cup \text{Lex}(\mathcal{O}_t) \cup \bigcup_{i=1}^n \text{Lex}(\mathcal{O}^{(i)}) \cup \text{range}(\text{docTable}) \cup \text{range}(\text{docAttr}),$$

so every $x \in \mathcal{X}$ is a natural-language string suitable for the embedding function, described in subsection 3.2.

3.2 Embedding-based Retrieval of Relevant Knowledge

To provide contextual knowledge about each table $r \in \mathcal{R}$, our pipeline performs embedding-based retrieval over the three knowledge sources (core ontology $\mathcal{O}_t$,

description of database schema $\mathcal{D}$, and external ontology repository $\mathcal{E}$ (Figure 2-(a), (b), (c)).

Embedding function. With the pre-trained model `all-mpnet-base-v2`, we define

$$\phi : \mathcal{X} \longrightarrow \mathbb{R}^d, \quad \mathbf{v}_x = \phi(x) \quad \text{for each } x \in \mathcal{X}$$

where d is the embedding dimension (768 for the chosen model). All relevant elements are first embedded into a unified semantic vector space.

All embeddings $\{\mathbf{v}_x | x \in \mathcal{X}\}$ are indexed using Faiss [15] for efficient approximate nearest neighbor (ANN) search based on cosine similarity. The similarity between two vectors $\mathbf{v}_x$ and $\mathbf{v}_y$ is defined as $\text{sim}(\mathbf{v}_x, \mathbf{v}_y) = \frac{\mathbf{v}_x \cdot \mathbf{v}_y}{\|\mathbf{v}_x\| \|\mathbf{v}_y\|}$.

Given a query embedding $\mathbf{v}_u = \phi(u)$ for the target table t or its column a , we retrieve the top- k most relevant elements from each source by computing:

$$\mathcal{N}_u^{\mathcal{O}_t} = \text{proj}_1 \left(\text{Top}_k \left\{ (x, \text{sim}(\mathbf{v}_u, \mathbf{v}_x)) \mid x \in \text{Lex}(\mathcal{O}_t) \right\} \right), \quad (1)$$

$$\mathcal{N}_u^{\mathcal{E}} = \text{proj}_1 \left(\text{Top}_k \left\{ (x, \text{sim}(\mathbf{v}_u, \mathbf{v}_x)) \mid x \in \text{Lex}(\mathcal{E}) \right\} \right), \quad (2)$$

$$\mathcal{N}_u^{\text{Doc}} = \text{proj}_1 \left(\text{Top}_k \left\{ (x, \text{sim}(\mathbf{v}_u, \mathbf{v}_x)) \mid \right. \right. \quad (3)$$

$$\left. x \in \text{range}(\text{docTable}) \cup \text{range}(\text{docAttr}) \right\} \right), \quad (4)$$

where $\text{proj}_1(x, \text{score})$ returns x . The retrieved information $\mathcal{N}_r^{\mathcal{O}_t} \cup \{\mathcal{N}_a^{\mathcal{O}_t} | a \in \mathcal{F}(r)\}$, $\mathcal{N}_r^{\mathcal{E}} \cup \{\mathcal{N}_a^{\mathcal{E}} | a \in \mathcal{F}(r)\}$, and $\mathcal{N}_r^{\text{Doc}} \cup \{\mathcal{N}_a^{\text{Doc}} | a \in \mathcal{F}(r)\}$ collectively forms the contextual foundation for constructing the Gen-LLM prompt (cf. Section 3.3).

3.3 LLM Prompt Construction and Ontology Generation

Given a relation $r \in \mathcal{R}$, we create a prompt, as depicted in Figure 2 and Figure 1-(e) for the *Gen-LLM*. The prompt includes the retrieved context (from table schema, textual documentation of the schema, external and core ontology), which we described in Section 3.2.

Then, we instruct the *Gen-LLM* to *generate an OWL ontology fragment* $\Delta\mathcal{O}_r$ for the specified table t . Our instructions include suitable object properties, data properties, OWL classes, and annotations that support the ontological modeling of the table. The prompt asks the *Gen-LLM* to (i) define OWL classes representing the table, (ii) define data properties for its columns, (iii) define object properties for foreign key relationships, (iv) include annotations such as provenance metadata, (v) use valid OWL syntax (e.g., one `rdfs:domain` and `rdfs:range` per property), and (vi) avoid using generic properties like “is” and encourages reuse of retrieved knowledge—by linking to or subclassing existing concepts from the core or external ontologies when appropriate. For each column $a \in \mathcal{F}(r)$, we instruct the *Gen-LLM* to either reuse an existing ontology property or generate a new property p_a that represents the column. We instruct the *Gen-LLM* to apply the following semantic constraints for the class C representing the table: (i) **Type constraint:** Add a universal restriction $C \sqsubseteq \forall p_a.\top$, where $\top$

Generate ontology elements with provenance annotations for database table '{data['table_name']}' based on:

[CONTEXT]

- Database Schema of the database '{data['schema_context']}'
- Take semantics from the Relevant Documents '{data['documents']}'
- Take semantics from the Existing Ontology Knowledge '{data['existing_ontology']}'

[INSTRUCTIONS]

1. Include these elements:
 - Classes (subclass of Thing)
 - Data properties with domain/range
 - Object properties with domain/range
2. Use only one `rdfs:domain` and one `rdfs:range` per property. If multiple options exist, select the most general or create a shared superclass.
3. Do not create a property named "is". Use `rdf:type` for instance membership, `rdfs:subClassOf` for class hierarchies, and `owl:sameAs` for instance equality.
4. Use this format example:

```

Class: '{data['table_name']}'
Annotations:
  prov:wasDerivedFrom
    <http://example.org/provenance/'{data['table_name']}'>

DataProperty:
  has_column_name
  domain: '{data['table_name']}'
  range string
  Annotations:
    prov:wasDerivedFrom
      <http://example.org/provenance/'{data['table_name']}'/column_name>

ObjectProperty:
  relates_to_table_domain '{data['table_name']}'
  range RelatedTable
  Annotations:
    prov:wasDerivedFrom
      <http://example.org/provenance/'{data['table_name']}'/fk_column>

```

Only output Manchester Syntax and nothing else. **[OUTPUT]**

Fig. 2: Prompt template used for ontology generation. Fixed delimiters such as [CONTEXT], [INSTRUCTIONS], and [OUTPUT] are included verbatim in the prompt to structure the LLM's input and guide its response. Placeholders in curly braces (e.g., {data['table_name']}) are dynamically replaced at runtime for each iteration with table-specific content.

is the range (either an `xsd` datatype or a class), (ii) **Cardinality constraint:** If a is declared NOT NULL, UNIQUE, or a primary key, also assert $C \sqsubseteq \exists p_a.1$ (or a qualified cardinality restriction), and (iii) **Property type selection:** If a participates in a foreign-key constraint, declare p_a as an *object property* with range equal to the referenced class; otherwise, declare it as a *datatype property* with an `xsd`-compatible range. Working in a retrieval-augmented environment, the *Gen-LLM* generates a delta ontology that expands the core ontology using

the provided schema and contextual information. This delta ontology includes a new class representing the table— or, in cases where the table implies several concepts, multiple classes—along with object properties that correspond to foreign keys and link to classes of referenced tables. It also includes attributes modeled as datatype properties or enumerations, as well as annotation assertions such as `rdfs:comment` that carry descriptive metadata.

3.4 Validation and Refinement of Generated Delta Ontology

To guarantee semantic soundness and compatibility with the overall schema and domain, each delta ontology $\Delta\mathcal{O}_i$ generated by the *Gen-LLM* is reviewed by a *Judge-LLM* (Figure 2-(g)). A prompt instructs the *Judge-LLM* to assess $\Delta\mathcal{O}_i$ based on several criteria, namely a) coherence with the current core ontology—e.g., avoiding the redefinition of concepts already introduced, such as generating a new class `ChemotherapyCycle` when one with the same name and semantics already exists in the core ontology, b) alignment with the input table schema (e.g., all significant columns or relationships are appropriately reflected), (c) syntactic validity and logical consistency of the delta ontology—ensuring that the OWL statements conform to the OWL 2 DL specification and can be parsed by standard reasoners (syntactic validity), and that no logical inconsistencies exist within the delta ontology, and d) clarity of class and property names and definitions, use of meaningful, self-explanatory, and domain-relevant terms with clear definitions and adequate documentation; avoid generic or ambiguous labels (e.g., “Entity1”, “PropertyA”). The *Judge-LLM* also confirms that all generated properties adhere to the OWL 2 DL profile, including accurate quantifiers, exactly one `rdfs:domain` and `rdfs:range`, and a consistent selection between object and datatype properties. It rejects or modifies fragments that would break the profile, introduce inconsistencies, or introduce unsatisfactory classes.

The *Judge-LLM* analyzes the $\Delta\mathcal{O}_i$ and then produces feedback, which may include qualitative assessments or ratings for each criterion, along with suggestions for improvements. If the *Judge-LLM* detects issues (e.g., an attribute not modeled or a class name that conflicts with an existing one), the delta ontology is updated. For high-impact ontology segments or when the automatic evaluation is unclear, a human expert may be consulted to manually examine and enhance the output. This human-in-the-loop step helps identify domain-specific details that might be outside the current scope of the *Gen-LLM* or guarantees adherence to established practices. After this validation step, we obtain a refined delta ontology fragment that is judged suitable in terms of correctness and quality.

3.5 Iterative Integration and Completion

After each cycle, the verified delta ontology $\Delta\mathcal{O}_i$ is merged into the core ontology $\mathcal{O}_i$ to form $\mathcal{O}_{i+1}$ (Figure 2-(h)). New classes, attributes, and axioms enlarge the core ontology while preserving external IRI references and reconciling duplicate concepts. The pipeline then follows foreign-key links to the next table, repeating the process until all tables, columns, and relationships have been covered.

The result is a complete OWL 2 DL ontology that mirrors the entire relational schema links entities via foreign-key constraints and incorporates external domain knowledge to meet ontology best-practice standards.

3.6 Competency Question Generation

We use CQs to evaluate the quality of the generated ontologies. CQs were generated using **Mistral-Small-24B-Instruct-2501** and verified by a domain expert. CQs are formulated as natural language questions and corresponding answers that reflect the type of information a specific SQL query is capable of retrieving. In our experiments, the Mistral model outperformed ChatGPT in generating high-quality CQs, as supported by prior comparative evaluations [31]. The superior performance of Mistral in this task highlights its suitability for structured-query-driven ontology validation and question-answering scenarios. We used the chain-of-thought prompt engineering technique [68] to generate these CQs, as shown in Appendix 6.2, Figure 3.

4 Experiments

4.1 Evaluation Databases

Relational Database To evaluate our approach, we conduct experiments on two distinct databases: a real-world clinical database obtained from a hospital and the publicly available ICU database from PhysioNet[52]. The **Real-World Database** comprises a liver cancer registry and contains 18 tables, 350 columns, and 15 foreign key relationships. Key tables include **chemotherapy**, **clavien_dindo**, and **medical_history**, which together capture essential information on treatment protocols, complication grading, and patient medical history. The **ICU Database** is sourced from Physio-Net and includes 32 tables, 559 columns, and 30 foreign keys. It models intensive care unit (ICU) admissions and includes clinically significant tables such as **admission_drugs**, **treatment**, and **respiratory_care**, representing pharmacological interventions, therapeutic procedures, and respiratory support, respectively. These diverse databases form a strong foundation to test our methodology’s generalizability and robustness.

4.2 External Ontology Repositories

We selected four biomedical ontologies from the BioPortal repository. These ontologies vary in domain coverage and structural complexity (Table 1). For example, the **International Classification of Diseases, Version 10** includes over 12,445 classes, providing broad coverage of disease categories.

4.3 Experimental Setup

Computational Resources Ontology generation was performed on high performance computing (HPC) infrastructure equipped with two NVIDIA A100 GPUs. The software environment was configured using Python version 3.10 to ensure compatibility with the machine learning frameworks used.

Name	#Classes	#Object Prop.	#Data Prop.	#Axioms
Cell Ontology	17,107	255	0	243,487
Human Disease Ontology	19,129	47	0	194,636
International Classification of Diseases, Version 10	12,445	233	1	17,139
Ontology for Nutritional Studies	4,735	73	3	27,326

Table 1: Statistics of selected biomedical ontologies from BioPortal, including the number of classes, properties, and axioms.

Database Documentation Collection To support the ontology mapping process, we employed GPT-4o (ChatGPT-4 Omni) to generate text description documents. These documents provide natural language explanations of various medical entities, including drugs, diseases, and clinical terminologies. The generated descriptions were reviewed by medical professionals to ensure clinical accuracy and domain relevance.

Large Language Models Multiple LLMs with varying quantization levels and parameter scales, accessible via the HuggingFace API, were employed to assess their efficacy in ontology generation. The models included Llama-3.1-70B, deepseek-llm-67b-chat-AWQ, and Mistral-Small-3.1-24B-Instruct-2503. Each model was evaluated on the quality and structure of its generated ontology.

4.4 Ontology Generation Methods

Baseline As a foundational benchmark, we evaluated a simple generation setup in which each *gen-LLM* received only the database schema as input. Ontologies were generated solely from schema information, without external context or retrieval augmentation. Our prompt design follows the structure proposed by [40], who used LLMs with similar zero-shot prompting to generate ontologies from natural language. In our case, we adapt their method to operate directly over structured database schemas instead of textual descriptions. This baseline serves as a reference for assessing the gains from our proposed RIGOR framework.

Non-Iterative Approach This approach extended the baseline by also supplying a sample external ontology alongside the schema. The model was prompted to generate a structurally and semantically similar ontology in a single pass—without iterative refinement or feedback.

RIGOR Framework Our framework utilizes a curated corpus of domain-specific documents pertaining to the medical field. Furthermore, the framework integrates a retrieval mechanism targeting External Ontologies, which are recognized as authoritative resources within the target domain. Relevant biomedical ontologies were retrieved via BioPortal’s REST API, a comprehensive repository of biomedical ontologies. These retrieved ontologies are incorporated into the retrieval pipeline, thereby providing enriched contextual grounding and improving the relevance and quality of the generated ontologies.

4.5 Evaluation Strategies

We employed six complementary evaluation techniques to assess the quality of the generated ontologies:

1. Syntax Validity Check: Each ontology was checked in Protégé for OWL 2 DL compliance.

2. Logical Consistency Check: We performed logical consistency checks using the HermiT reasoner within the Protégé ontology editor.

3. Criteria-Based Evaluation (Modeling Pitfalls): We used the OOPS! (Ontology Pitfall Scanner) web interface to detect common modeling pitfalls in our ontologies [53,34]. OOPS! scans for 41 predefined pitfalls, categorized as *critical*, *important*, or *minor* based on their impact on ontology quality.

4. Structural Analysis: We report structural metrics of the generated ontologies using the Protégé ontology editor, including the number of classes, object properties, data properties, and axioms. These metrics provide a quantitative overview of the ontology’s size and complexity.

5. Semantic Coverage of Database Schema: To evaluate how well the generated ontology represents the underlying database schema, we performed a semantic similarity analysis between ontology class names and table column names. Using the `all-MiniLM-L6-v2` sentence transformer, we embedded both sets and computed cosine similarity scores. We then calculated a *class coverage rate*, defined as the percentage of ontology classes with at least one column name above a similarity threshold of 0.55. This threshold was empirically tuned and aligns with prior work [18].

6. Ontology Quality through CQ Performance: This strategy evaluates the quality of the generated ontologies based on their ability to answer CQs. The CQs themselves were generated using `Mistral-Small-24B-Instruct-2501`, as described in Section 3.6, where we outline the CQ generation pipeline and prompting engineering technique.

For evaluation, we employed a separate model, `Llama-3.1-8B-Instruct` (*Judge-LLM*) which scored the answers derived from the ontologies on a scale from 0 to 5. Each delta ontology was evaluated independently, and scores were then aggregated to report the overall CQ performance for each generation method. Each ontology was assessed across a set of predefined quality dimensions [52]. The scoring prompt, tuned through prompt refinement, is shown in Appendix 6.2, Figure 4. The evaluation dimensions included: (a) **Accuracy:** Correctness of domain representation and alignment with CQs. (b) **Completeness:** Coverage of essential concepts and relationships. (c) **Conciseness:** Absence of redundancy in the generated ontology. (d) **Adaptability:** Extensibility of the generated ontology. (e) **Clarity:** Readability, clear definitions, and quality of documentation. (f) **Consistency:** Logical coherence, valid relations, and proper hierarchies.

4.6 Results

Upon inspecting the generated outputs, baseline and non-iterative methods often failed to produce valid ontologies—frequently yielding invalid Turtle syntax

or irrelevant outputs (e.g., SQL). As a result, it was not possible to merge delta ontologies or evaluate them using structural, logical, or semantic strategies. Consequently, Strategies 1–5 report results only for **RIGOR**-generated outputs. In contrast, Strategy 6 (Ontology Quality through CQ Performance) was applied to all generation methods, as it evaluates each delta ontology independently without requiring ontology merging or syntactic validity.

1. Syntax Validity Check: All **RIGOR**-generated ontologies were parsed in Protégé and conformed to the OWL 2 DL profile, ensuring syntactic validity.

2. Logical Consistency Check: All **RIGOR**-generated ontologies were deemed consistent by the HermiT reasoner.

3. Criteria-Based Evaluation (Modeling Pitfalls): Table 3 presents only *critical* and *important* modeling pitfalls as categorized by OOPS!, excluding low-severity pitfalls such as missing annotations. Overall, the ontologies demonstrated sound modeling practices. The most frequent issue was (P19) (Multiple domains or ranges), which may arise in auto-generated ontologies when property reuse is not well defined in the prompt. A few cases of (P34) (Untyped class) and (P12/P30) (missing equivalence declarations) were detected, but these reflect refinement opportunities rather than structural errors. The absence of critical pitfalls—such as defining wrong relations (P05, P27, P28, P29, P31) or cycles in hierarchies (P06)—indicates that the generated ontologies adhere to core principles of formal ontology modeling.

4. Structural Analysis: We compared structural metrics across ontologies generated by different *Gen-LLMs* under the **RIGOR** framework. As shown in Table 4, DeepSeek consistently produced the most structurally rich ontologies, with the highest counts of classes and axioms. Mistral generated the smallest ontologies in terms of class count, though it occasionally had high data property counts, especially in the ICU domain. Overall, Ontologies from the ICU database were structurally more complex than those from the real-world database, indicating a correlation between database size and the number of classes, properties, and axioms, consistent with the ICU database’s larger schema in Section 4.1.

5. Semantic Coverage of Database Schema: This analysis was performed on the **chemotherapy** ontology—one of the delta ontologies generated using the **RIGOR** framework and LLaMA 3.1 using the real-world database. In total, 21 out of 29 ontology classes were matched to column names, yielding a *class coverage* of 72.4%, indicating strong semantic alignment and showing how well the local features of the database table are represented in the corresponding delta ontology. Figure 5 visualizes the **chemotherapy** ontology and shows the heatmap of class-to-column alignment. Unmatched table columns (e.g., `patient_id`) refer to concepts modeled in other delta ontologies within the full ontology; for instance, `patient_id` appears in the **patient** ontology and is omitted from the **chemotherapy** ontology by the LLM to avoid redundancy. Unmatched ontology classes (e.g., `TreatmentPlan`) were derived from external ontology repositories or retrieved documentation as part of our RAG pipeline.

6. Ontology Quality through CQ Performance: Table 2 shows that the **RIGOR** method consistently outperformed Baseline and Non-Iterative ap-

<i>gen-LLM</i>	Real-world database			ICU database from PhysioNet		
	Baseline	Non-Iterative	RIGOR	Baseline	Non-Iterative	RIGOR
Llama 3.1	1.5 $\pm$.31	3.5 $\pm$.31	4.5 $\pm$.20	1.6 $\pm$.34	3.7 $\pm$.51	4.6 $\pm$.12
Mistral 3.1	1.6 $\pm$.23	3.4 $\pm$.67	4.2 $\pm$.22	1.3 $\pm$.41	3.7 $\pm$.36	4.3 $\pm$.15
DeepSeek 67B	1.4 $\pm$.31	3.3 $\pm$.38	4.6 $\pm$.11	1.5 $\pm$.15	3.5 $\pm$.27	4.5 $\pm$.13

Table 2: Average scores (0-5) across six ontology quality dimensions (accuracy, completeness, conciseness, adaptability, clarity, and consistency) for each *gen-LLM* and generation method on two databases. Values indicate mean $\pm$ standard deviation. Higher scores indicate better performance.

Code	Pitfall Description	Real-world database			ICU database from PhysioNet		
		Llama 3.1	Mistral 3.1	DeepSeek 67B	Llama 3.1	Mistral 3.1	DeepSeek 67B
P03	Creating the relationship "is"	1	0	0	0	0	0
P10	Missing disjointness	1	1	1	1	1	1
P12	Equivalent properties not declared	0	1	0	6	10	4
P19	Multiple domains or ranges	23	11	25	31	22	54
P30	Equivalent classes not declared	3	0	4	0	0	2
P34	Untyped class	3	2	21	7	7	17
P41	No license declared	1	1	1	1	1	1

Table 3: Pitfalls detected by OOPS! in **RIGOR**-generated ontologies using three *gen-LLMs* across two databases.

<i>gen-LLM</i>	Real-world database				ICU database from PhysioNet			
	#Classes	#Object Prop.	#Data Prop.	#Axioms	#Classes	#Object Prop.	#Data Prop.	#Axioms
Llama 3.1	284	76	145	1,390	359	89	182	1,946
Mistral 3.1	30	10	115	600	70	18	339	1,666
DeepSeek 67B	287	68	522	1,584	389	120	171	2,070

Table 4: Statistics of **RIGOR**-generated ontologies using three *gen-LLMs* across two databases, including the number of classes, properties, and axioms.

proaches across all models and databases. Detailed scores across all dimensions are reported in Tables 6–7. The highest CQ scores were achieved by DeepSeek 67B on the real-world database (4.6 $\pm$ 0.11) and Llama 3.1 on ICU (4.6 $\pm$ 0.12). Mistral 3.1 performed slightly lower but still benefitted from RIGOR.

5 Conclusions

We presented **RIGOR**, an iterative hybrid-recursive RAG pipeline that converts relational schemas into provenance-rich *OWL 2 DL* ontologies with minimal human effort. By combining dense retrieval from the evolving core ontology, textual documentation, and large external ontology repositories with a *Gen-LLM/Judge-LLM* loop, RIGOR produces table-level delta ontologies that are validated, refined, and incrementally merged.

Experiments on a real-world liver-cancer registry and the public ICU database show that RIGOR consistently surpasses baseline and non-iterative variants, achieving mean CQ scores above 4.5/5, yielding logically consistent ontologies, and exhibiting few critical modeling pitfalls. These results show that retrieval-guided LLMs can generate semantically rich, standards-compliant ontologies that faithfully mirror complex relational schemas.

Acknowledgement

Mojtaba Nayyeri gratefully acknowledges support from the ATLAS project, funded by the German Federal Ministry of Education and Research (Bundesministerium für Bildung und Forschung, BMBF). The authors gratefully acknowledge the computing time provided on the HPC HoreKa by the National HPC Center at KIT (NHR@KIT). This center is jointly supported by the Federal Ministry of Education and Research and the Ministry of Science, Research, and the Arts of Baden-Württemberg as part of the National High-Performance Computing (NHR) joint funding program (<https://www.nhr-verein.de/en/our-partners>). HoreKa is partly funded by the German Research Foundation (DFG).

Supplemental Material Statement

Source code is available in "RIGOR.zip", including core ontologies, RAG documents, appendix, external ontologies, SQL schemas, generated CQs, experimental results, and specific scripts for the RIGOR framework, baseline, "non-iterative" approach, requirements, and semantic evaluation.

References

1. Abolhasani, M.S., Pan, R.: Leveraging llm for automated ontology extraction and knowledge graph generation. arXiv preprint arXiv:2412.00608 (2024)
2. Alharbi, R., Tamma, V., Grasso, F., Payne, T.R.: An experiment in retrofitting competency questions for existing ontologies. In: Hong, J., Park, J.W. (eds.) Proceedings of the 39th ACM/SIGAPP Symposium on Applied Computing, SAC 2024, Avila, Spain, April 8-12, 2024. pp. 1650–1658. ACM (2024). <https://doi.org/10.1145/3605098.3636053>, <https://doi.org/10.1145/3605098.3636053>
3. Antia, M., Keet, C.M.: Automating the generation of competency questions for ontologies with agocqs. In: Ortiz-Rodríguez, F., Villazón-Terrazas, B., Tiwari, S., Bobed, C. (eds.) Knowledge Graphs and Semantic Web - 5th Iberoamerican Conference and 4th Indo-American Conference, KGSWC 2023, Zaragoza, Spain, November 13-15, 2023, Proceedings. Lecture Notes in Computer Science, vol. 14382, pp. 213–227. Springer (2023). https://doi.org/10.1007/978-3-031-47745-4_16, https://doi.org/10.1007/978-3-031-47745-4_16
4. Arenas, M., Bertails, A., Prud'hommeaux, E., Sequeda, J., et al.: A direct mapping of relational data to rdf. W3C recommendation **27**, 1–11 (2012)
5. Astrova, I.: Reverse engineering of relational databases to ontologies. In: European Semantic Web Symposium. pp. 327–341. Springer (2004)
6. Babaei Giglou, H., D'Souza, J., Auer, S.: Llms4ol: Large language models for ontology learning. In: International Semantic Web Conference. pp. 408–427. Springer (2023)
7. Babaei Giglou, H., D'Souza, J., Auer, S.: Llms4ol: Large language models for ontology learning. In: International Semantic Web Conference. pp. 408–427. Springer (2023)

8. Bakker, R.M., Di Scala, D.L., de Boer, M.H.: Ontology learning from text: an analysis on llm performance. In: *Proceedings of the 3rd NLP4KGC International Workshop on Natural Language Processing for Knowledge Graph Creation, colocated with Semantics*. pp. 17–19 (2024)
9. Ben Mahria, B., Chaker, I., Zahi, A.: A novel approach for learning ontology from relational database: from the construction to the evaluation. *Journal of Big Data* **8**(1), 25 (2021)
10. Câmara, V., Mendonca-Neto, R., Silva, A., Cordovil, L.: A large language model approach to sql-to-text generation. In: *2024 IEEE International Conference on Consumer Electronics (ICCE)*. pp. 1–4. IEEE (2024)
11. Caulfield, J.H., Hegde, H., Emonet, V., Harris, N.L., Joachimiak, M.P., Matentzoglou, N., Kim, H., Moxon, S., Reese, J.T., Haendel, M.A., et al.: Structured prompt interrogation and recursive extraction of semantics (spires): A method for populating knowledge bases using zero-shot learning. *Bioinformatics* **40**(3), btac104 (2024)
12. Ciroku, F., de Berardinis, J., Kim, J., Meroño-Peñuela, A., Presutti, V., Simperl, E.: Revont: Reverse engineering of competency questions from knowledge graphs via language models. *J. Web Semant.* **82**, 100822 (2024). <https://doi.org/10.1016/J.WEBSEM.2024.100822>, <https://doi.org/10.1016/j.websem.2024.100822>
13. Consortium, W.W.W., et al.: R2rml: Rdb to rdf mapping language. WWW (2012)
14. Doumanas, D., Soularidis, A., Spiliotopoulos, D., Vassilakis, C., Kotis, K.: Fine-tuning large language models for ontology engineering: A comparative analysis of gpt-4 and mistral. *Applied Sciences* **15**(4), 2146 (2025)
15. Douze, M., Guzhva, A., Deng, C., Johnson, J., Szilvasy, G., Mazaré, P.E., Lomeli, M., Hosseini, L., Jégou, H.: The faiss library. *arXiv preprint arXiv:2401.08281* (2024)
16. Fathallah, N., Das, A., Giorgis, S.D., Poltronieri, A., Haase, P., Kovriguina, L.: Neon-gpt: a large language model-powered pipeline for ontology learning. In: *European Semantic Web Conference*. pp. 36–50. Springer (2024)
17. Galkin, M., Yuan, X., Mostafa, H., Tang, J., Zhu, Z.: Towards foundation models for knowledge graph reasoning. In: *The Twelfth International Conference on Learning Representations* (2024)
18. Galli, C., Donos, N., Calciolari, E.: Performance of 4 pre-trained sentence transformer models in the semantic query of a systematic review dataset on peri-implantitis. *Information* **15**(2), 68 (2024)
19. Gao, Y., Xiong, Y., Gao, X., Jia, K., Pan, J., Bi, Y., Dai, Y., Sun, J., Wang, M., Wang, H.: Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997* (2023)
20. Grattafiori, A., Dubey, A., Jauhri, A., Pandey, A., Kadian, A., Al-Dahle, A., Letman, A., Mathur, A., Schelten, A., Vaughan, A., et al.: The llama 3 herd of models. *arXiv preprint arXiv:2407.21783* (2024)
21. Gruninger, M.: Methodology for the design and evaluation of ontologies. In: *Proc. IJCAI'95, Workshop on Basic Ontological Issues in Knowledge Sharing* (1995)
22. Gu, J., Jiang, X., Shi, Z., Tan, H., Zhai, X., Xu, C., Li, W., Shen, Y., Ma, S., Liu, H., Wang, Y., Guo, J.: A survey on llm-as-a-judge. *CoRR* **abs/2411.15594** (2024). <https://doi.org/10.48550/ARXIV.2411.15594>, <https://doi.org/10.48550/arXiv.2411.15594>
23. Gu, J., Jiang, X., Shi, Z., Tan, H., Zhai, X., Xu, C., Li, W., Shen, Y., Ma, S., Liu, H., et al.: A survey on llm-as-a-judge. *arXiv preprint arXiv:2411.15594* (2024)

24. Guo, D., Yang, D., Zhang, H., Song, J., Zhang, R., Xu, R., Zhu, Q., Ma, S., Wang, P., Bi, X., et al.: Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. arXiv preprint arXiv:2501.12948 (2025)
25. Horridge, M., Patel-Schneider, P.F.: Owl 2 web ontology language manchester syntax. <https://www.w3.org/TR/owl2-manchester-syntax/> (2012)
26. Jiménez-Ruiz, E., Kharlamov, E., Zheleznyakov, D., Horrocks, I., Pinkel, C., Skjæveland, M.G., Thorstensen, E., Mora, J.: Bootox: Practical mapping of rdfs to owl 2. In: The Semantic Web-ISWC 2015: 14th International Semantic Web Conference, Bethlehem, PA, USA, October 11-15, 2015, Proceedings, Part II 14. pp. 113–132. Springer (2015)
27. Keet, C.M., Mahlaza, Z., Antia, M.: Claro: A controlled language for authoring competency questions. In: Garoufallou, E., Fallucchi, F., Luca, E.W.D. (eds.) Metadata and Semantic Research - 13th International Conference, MTSR 2019, Rome, Italy, October 28-31, 2019, Revised Selected Papers. Communications in Computer and Information Science, vol. 1057, pp. 3–15. Springer (2019). https://doi.org/10.1007/978-3-030-36599-8_1, https://doi.org/10.1007/978-3-030-36599-8_1
28. Knoblock, C.A., Szekely, P., Ambite, J.L., Goel, A., Gupta, S., Lerman, K., Muslea, M., Taheriyani, M., Mallick, P.: Semi-automatically mapping structured sources into the semantic web. In: Extended semantic web conference. pp. 375–390. Springer (2012)
29. Kohler, J., Lange, M., Hofstadt, R., Schulze-Kremer, S.: Logical and semantic database integration. In: Proceedings IEEE International Symposium on Bio-Informatics and Biomedical Engineering. pp. 77–80. IEEE (2000)
30. Kommineni, V.K., König-Ries, B., Samuel, S.: Towards the automation of knowledge graph construction using large language models. Journal Name (2024)
31. Kommineni, V.K., König-Ries, B., Samuel, S.: From human experts to machines: An llm supported approach to ontology and knowledge graph construction (2024), <https://arxiv.org/abs/2403.08345>
32. Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.t., Rocktäschel, T., et al.: Retrieval-augmented generation for knowledge-intensive nlp tasks. Advances in neural information processing systems **33**, 9459–9474 (2020)
33. Lippolis, A.S., Saeedizade, M.J., Keskiärrkkä, R., Zuppiroli, S., Ceriani, M., Gangemi, A., Blomqvist, E., Nuzzolese, A.G.: Ontology generation using large language models. arXiv preprint arXiv:2503.05388 (2025)
34. Lippolis, A.S., Saeedizade, M.J., Keskiärrkkä, R., Zuppiroli, S., Ceriani, M., Gangemi, A., Blomqvist, E., Nuzzolese, A.G.: Ontology generation using large language models. arXiv preprint arXiv:2503.05388 (2025)
35. Liu, M.X., Liu, F., Fiannaca, A.J., Koo, T., Dixon, L., Terry, M., Cai, C.J.: "we need structured output": Towards user-centered constraints on large language model output. In: Extended Abstracts of the CHI Conference on Human Factors in Computing Systems. pp. 1–9 (2024)
36. Liu, Y., Li, D., Wang, K., Xiong, Z., Shi, F., Wang, J., Li, B., Hang, B.: Are llms good at structured outputs? a benchmark for evaluating structured output capabilities in llms. Information Processing & Management **61**(5), 103809 (2024)
37. Liu, Z., Gan, C., Wang, J., Zhang, Y., Bo, Z., Sun, M., Chen, H., Zhang, W.: Ontotune: Ontology-driven self-training for aligning large language models. In: THE WEB CONFERENCE 2025 (2025)

38. Lo, A., Jiang, A.Q., Li, W., Jamnik, M.: End-to-end ontology learning with large language models. In: The Thirty-eighth Annual Conference on Neural Information Processing Systems (2024)
39. Ma, C., Molnár, B.: Ontology learning from relational database: Opportunities for semantic information integration. *Vietnam Journal of Computer Science* **9**(01), 31–57 (2022)
40. Mateiu, P., Groza, A.: Ontology engineering with large language models. In: 2023 25th International Symposium on Symbolic and Numeric Algorithms for Scientific Computing (SYNASC). pp. 226–229. IEEE (2023)
41. de Medeiros, L.F., Priyatna, F., Corcho, O.: Mirror: Automatic r2rml mapping generation from relational databases. In: International conference on web engineering. pp. 326–343. Springer (2015)
42. Meersman, R.: Ontologies and databases: More than a fleeting resemblance. In: A. d’Atri and M. Missikoff (eds), *Proceedings of the OES/SEO 2001 Rome Workshop*, LUISS Publications, Rome (2001) (2001)
43. Meyer, L.P., Frey, J., Junghanns, K., Brei, F., Bulert, K., Gründer-Fahrer, S., Martin, M.: Developing a scalable benchmark for assessing large language models in knowledge graph engineering. *arXiv preprint arXiv:2308.16622* (2023)
44. Mihindukulasooriya, N., Tiwari, S., Enguix, C.F., Lata, K.: Text2kgbench: A benchmark for ontology-driven knowledge graph generation from text. In: International semantic web conference. pp. 247–265. Springer (2023)
45. Mihindukulasooriya, N., Tiwari, S., Enguix, C.F., Lata, K.: Text2kgbench: A benchmark for ontology-driven knowledge graph generation from text. In: International semantic web conference. pp. 247–265. Springer (2023)
46. Motik, B., Horrocks, I., Sattler, U.: Bridging the gap between owl and relational databases. In: *Proceedings of the 16th international conference on World Wide Web*. pp. 807–816 (2007)
47. Norouzi, S.S., Barua, A., Christou, A., Gautam, N., Eells, A., Hitzler, P., Shimizu, C.: Ontology population using llms. In: *Handbook on Neurosymbolic AI and Knowledge Graphs*, pp. 421–438. IOS Press (2025)
48. Norouzi, S.S., Barua, A., Christou, A., Gautam, N., Eells, A., Hitzler, P., Shimizu, C.: Ontology population using llms. In: *Handbook on Neurosymbolic AI and Knowledge Graphs*, pp. 421–438. IOS Press (2025)
49. Passant, A., Gandon, F., Alani, H., Spanos, D.E., Stavrou, P., Mitrou, N.: Bringing relational databases into the semantic web: A survey. *Semantic Web* **3**(2), 169–209 (2012)
50. Pinkel, C., Binnig, C., Jiménez-Ruiz, E., Kharlamov, E., May, W., Nikolov, A., Sasa Bastinos, A., Skjæveland, M.G., Solimando, A., Taheriyani, M., et al.: Rodi: Benchmarking relational-to-ontology mapping generation quality. *Semantic Web* **9**(1), 25–52 (2017)
51. Pinkel, C., Binnig, C., Jiménez-Ruiz, E., Kharlamov, E., Nikolov, A., Schwarte, A., Heupel, C., Kraska, T.: Incmap: A journey towards ontology-based data integration. *Lecture Notes in Informatics (LNI), Proceedings-Series of the Gesellschaft für Informatik (GI)* **265**, 145–164 (2017)
52. Pollard, T., Johnson, A., Raffa, J., Celi, L., Mark, R., Badawi, O.: The eicu collaborative research database, a freely available multi-center database for critical care research. *Scientific Data* **5**, 180178 (09 2018). <https://doi.org/10.1038/sdata.2018.178>
53. Poveda-Villalón, M., Gómez-Pérez, A., Suárez-Figueroa, M.C.: Oops! (ontology pitfall scanner!): An on-line tool for ontology evaluation. *International*

- Journal on Semantic Web and Information Systems (IJSWIS) **10**(2), 7–34 (2014). <https://doi.org/10.4018/IJSWIS.2014040102>, <https://doi.org/10.4018/ijswis.2014040102>
54. Qiang, Z., Wang, W., Taylor, K.: Agent-om: Leveraging llm agents for ontology matching. arXiv preprint arXiv:2312.00326 (2023)
 55. Rebboud, Y., Tailhardat, L., Lisena, P., Troncy, R.: Can llms generate competency questions? In: Merono-Penuela, A., Corcho, Ó., Groth, P., Simperl, E., Tamma, V., Nuzzolese, A.G., Poveda-Villalón, M., Sabou, M., Presutti, V., Celino, I., Revenko, A., Raad, J., Sartini, B., Lisena, P. (eds.) The Semantic Web: ESWC 2024 Satellite Events - Hersonissos, Crete, Greece, May 26-30, 2024, Proceedings, Part I. Lecture Notes in Computer Science, vol. 15344, pp. 71–80. Springer (2024). https://doi.org/10.1007/978-3-031-78952-6_7, https://doi.org/10.1007/978-3-031-78952-6_7
 56. Robinson, J., Ranjan, R., Hu, W., Huang, K., Han, J., Dobles, A., Fey, M., Lenssen, J.E., Yuan, Y., Zhang, Z., et al.: Relbench: A benchmark for deep learning on relational databases. Advances in Neural Information Processing Systems **37**, 21330–21341 (2024)
 57. Roumeliotis, K.I., Tselikas, N.D.: Chatgpt and open-ai models: A preliminary review. Future Internet **15**(6), 192 (2023)
 58. Sadruddin, S., D’Souza, J., Poupaki, E., Watkins, A., Giglou, H.B., Rula, A., Karasulu, B., Auer, S., Mackus, A., Kessels, E.: Llms4schemadiscovery: A human-in-the-loop workflow for scientific schema mining with large language models. arXiv preprint arXiv:2504.00752 (2025)
 59. Saeedizade, M.J., Blomqvist, E.: Navigating ontology development with large language models. In: European Semantic Web Conference. pp. 143–161. Springer (2024)
 60. Sequeda, J.F., Arenas, M., Miranker, D.P.: On directly mapping relational databases to rdf and owl. In: Proceedings of the 21st international conference on World Wide Web. pp. 649–658 (2012)
 61. Sharma, K., Kumar, P., Li, Y.: Og-rag: Ontology-grounded retrieval-augmented generation for large language models. arXiv preprint arXiv:2412.15235 (2024)
 62. Shimizu, C., Hitzler, P.: Accelerating knowledge graph and ontology engineering with large language models. Journal of Web Semantics p. 100862 (2025)
 63. Val-Calvo, M., Aranguren, M.E., Mulero-Hernández, J., Almagro-Hernández, G., Deshmukh, P., Bernabé-Díaz, J.A., Espinoza-Arias, P., Sánchez-Fernández, J.L., Mueller, J., Fernández-Breis, J.T.: Ontogenix: Leveraging large language models for enhanced ontology engineering from datasets. Information Processing & Management **62**(3), 104042 (2025)
 64. Vergoulis, A., et al.: Data governance in the era of the web of data: generate, manage, preserve, share and protect resources in the web of data. Tech. rep., Technical Report (2014)
 65. Volz, R., Handschuh, S., Staab, S., Stojanovic, L., Stojanovic, N.: Unveiling the hidden bride: deep annotation for mapping and migrating legacy data to the semantic web. J. Web Semant. **1**(2), 187–206 (2004). <https://doi.org/10.1016/J.WEBSEM.2003.11.005>, <https://doi.org/10.1016/j.websem.2003.11.005>
 66. W3C OWL Working Group: Owl 2 web ontology language profiles (second edition). <http://www.w3.org/TR/owl2-profiles/> (2012). w3C Recommendation
 67. Wache, H., Voegelé, T., Visser, U., Stuckenschmidt, H., Schuster, G., Neumann, H., Hübner, S.: Ontology-based integration of information—a survey of existing approaches. In: Ois@ ijcai (2001)

68. Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E.H., Le, Q.V., Zhou, D.: Chain-of-thought prompting elicits reasoning in large language models. In: Koyejo, S., Mohamed, S., Agarwal, A., Belgrave, D., Cho, K., Oh, A. (eds.) *Advances in Neural Information Processing Systems 35: Annual Conference on Neural Information Processing Systems 2022, NeurIPS 2022, New Orleans, LA, USA, November 28 - December 9, 2022* (2022), http://papers.nips.cc/paper_files/paper/2022/hash/9d5609613524ecf4f15af0f7b31abca4-Abstract-Conference.html
69. Wu, Z., Pan, S., Chen, F., Long, G., Zhang, C., Yu, P.S.: A comprehensive survey on graph neural networks. *IEEE transactions on neural networks and learning systems* **32**(1), 4–24 (2020)
70. Xiong, B., Staab, S.: From tokens to lattices: Emergent lattice structures in language models. In: *The Thirteenth International Conference on Learning Representations* (2025), <https://openreview.net/forum?id=md9qolJwLl>
71. Zhao, W.X., Zhou, K., Li, J., Tang, T., Wang, X., Hou, Y., Min, Y., Zhang, B., Zhang, J., Dong, Z., et al.: A survey of large language models. *arXiv preprint arXiv:2303.18223* (2023)
72. Zhou, H., Nova, A., Larochelle, H., Courville, A., Neyshabur, B., Sedghi, H.: Teaching algorithmic reasoning via in-context learning. *arXiv preprint arXiv:2211.09066* (2022)

6 Appendix

This appendix provides supplementary material supporting the methods, evaluation, and implementation described in the main paper. It is organized as follows:

6.1 Summary of Symbols

Table 5 summarizes the key formal symbols used throughout the paper to describe database schemas, ontology structures, and transformation steps. This section serves as a reference for interpreting the technical framework introduced in the methodology.

Symbol	Definition
$\mathcal{S} = (\mathcal{R}, \mathcal{K}, \mathcal{F})$	Relational database schema
$\mathcal{R}$	Set of relations (tables) in the database
$\mathcal{K}(r)$	Primary key attributes of relation r
$\mathcal{F}_k(r)$	Foreign key attributes of relation r
$\mathcal{F}(r)$	Attributes (columns) of relation r
$\text{DocTable}(r)$	Textual description of table r
$\text{DocCol}(r, f)$	Textual description of attribute f in table r
$\mathcal{O}_t = (\mathcal{C}_t, \mathcal{P}_t, \mathcal{A}_t)$	Core ontology at iteration t
$\mathcal{C}_t$	Set of OWL classes defined so far
$\mathcal{P}_t$	Set of OWL properties (object and data)
$\mathcal{A}_t$	Set of OWL axioms and annotations
$\Delta\mathcal{O}_r$	Delta-ontology fragment generated for relation r
$\mathcal{O}_{\text{final}}$	Final integrated OWL ontology after processing all relations

Table 5: Formalization of relational database, textual descriptions, and ontology structures.

6.2 Prompts

This section presents the prompts used in two core stages of our pipeline: (a) generating CQs from relational tables, and (b) evaluating ontology fragments based on those questions using an LLM-based scoring mechanism.

Given the SQL table schema for table '{table_name}':
 Generate 5 competency questions (CQs) that this table's ontology should answer.
 For each question, also provide:

- A short answer explaining how the ontology would answer it using the table schema.

Think step-by-step based on the data and relationships.
 Competency Questions for Table '{table_name}'

****1. [Question]****
 - ****Answer**:** [Explanation]

Continue similarly for 5 questions.
 Do not include any additional commentary or explanation outside the format.

Fig. 3: (a) The CQs Generation prompt: The prompt used to guide the LLM in generating CQs relevant to each database table. It was refined through empirical tuning of prompt engineering techniques to encourage structured, ontology-aligned outputs. Fixed delimiters such as ****1. [Question]**** and ****Answer**:** are included verbatim to structure the LLM's response as paired CQs and explanations. The placeholder '{table_name}' is dynamically substituted at runtime for each table to guide the model toward schema-relevant, ontology-aligned question generation.

You are an ontology evaluation expert. Analyze this ontology fragment and provide a numerical score (0-5) for metric. Criteria: '{get_metric_criteria(metric)}'
 Return ONLY a numerical score between (0.0-5.0) with one decimal place

Competency Questions: '{context['questions']}'

Ontology Fragment: '{context['ontology_chunk']}'

Database Schema: '{context['schema']}'

Format your response as: "Score: X.X"

Fig. 4: (b) Ontology quality evaluation through CQ performance prompt: The prompt used to evaluate the quality of ontology fragments with respect to specific metrics (e.g., accuracy, completeness). The Judge-LLM receives the CQs, ontology fragment, and corresponding table schema, and returns a numeric score (0–5) based on a specified evaluation criterion. Placeholders like '{context['questions']}' are dynamically replaced at runtime with evaluation-specific content.

6.3 Implementation Details:

Provides code-level information used to run Huggingface models in our experiments, including model setup, decoding parameters, and inference configuration.

```
def setup_huggingface_model(model_name="mistralai/Mistral-8B-Instruct-2410"):
    tokenizer = AutoTokenizer.from_pretrained(model_name)
```



```

model = AutoModelForCausalLM.from_pretrained(model_name)

hf_pipeline = pipeline(
    "text-generation",
    model=model,
    tokenizer=tokenizer,
    device_map="auto",
    trust_remote_code=True,
    torch_dtype=torch.bfloat16,
    max_new_tokens=2000, #originally 1000
    do_sample=True,
    temperature=0.7
)

return HuggingFacePipeline(pipeline=hf_pipeline)

```

6.4 Detailed Evaluation Results

This section presents both quantitative and structural evaluation results supporting the main findings of the paper.

(a) Judge-LLM Evaluation Details: Tables 6-7 reports detailed CQ performance scores across six evaluation dimensions—accuracy, completeness, conciseness, adaptability, clarity, and consistency—for each *gen-LLM* and generation strategy on the real-world and ICU databases, respectively. These scores were generated using the Judge-LLM, which evaluated each delta ontology fragment in its local context. The reported values represent aggregated scores across all delta ontologies within each database.

<i>gen-LLMs</i>	Accuracy	Completeness	Conciseness	Adaptability	Clarity	Consistency	Overall
LLama	4.08	4.52	4.5	4.53	4.5	4.78	4.4
Mistral	4.12	4.35	4.23	4.15	4.21	4.14	4.2
Deepseek	4.49	4.5	4.5	4.5	4.5	4.79	4.6

Table 6: CQ Performance comparison of *gen-LLMs* across Baseline, Non-Iterative, and **RIGOR** methods on real-world database across various metrics.

<i>gen-LLMs</i>	Accuracy	Completeness	Conciseness	Adaptability	Clarity	Consistency	Overall
LLama	4.5	4.54	4.33	4.5	4.43	4.73	4.5
Mistral	4.2	4.23	4.24	4.19	4.12	4.11	4.2
Deepseek	4.5	4.53	4.33	4.5	4.41	4.73	4.5

Table 7: CQ Performance comparison of *gen-LLMs* across Baseline, Non-Iterative, and **RIGOR** methods on ICU database across various metrics.

(b) Delta Ontology Visualization and Semantic Alignment: Figure 5 complements this analysis by illustrating a specific delta ontology (**chemotherapy** ontology— generated using the RIGOR framework and LLaMA 3.1 on the Real-world database) and its semantic alignment with the database schema. Part (a) visualizes the structure of the ontology. Part (b) shows the semantic similarity heatmap between ontology class names and column names of the corresponding table, highlighting the ontology’s schema-level coverage.

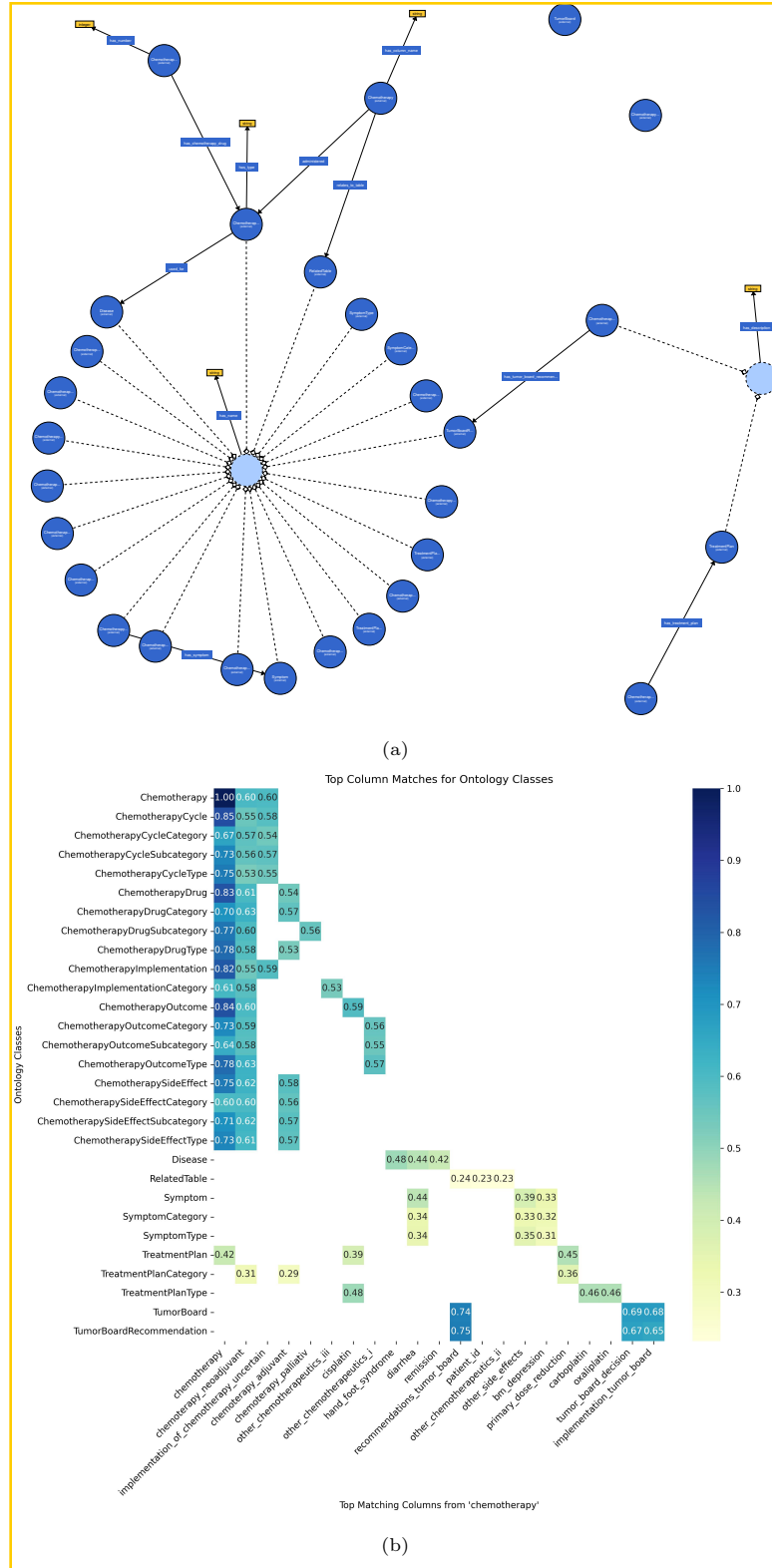


Fig. 5: (a) Visual representation of the **chemotherapy** delta ontology structure generated by Llama 3.1 from **RIGOR** from real-world database, generated using WebVOWL 1.1.7. (b) Semantic alignment between ontology classes and database table column names based on embedding-based similarity.